AI ASSISTED CODING
ASSIGNMENT NO:13.5

Name:Mohammed Rahil Raza

Ht.no:2303A51433

bt.no:21

Task Description #1 (Refactoring – Removing Global Variables)

• Task: Use AI to eliminate unnecessary global variables from the]code.

• Instructions:

o Identify global variables used across functions.

o Refactor the code to pass values using function parameters.

o Improve modularity and testability.

• Sample Legacy Code:

```
rate = 0.1
def calculate_interest(amount):   return amount * rate
print(calculate_interest(1000))
```

code screenshot:

```python
def calculate_interest(amount, rate):
    return amount * rate

# Example usage
interest_rate = 0.1

print(calculate_interest(1000, interest_rate))

# Refactored Code using a Class to Encapsulate the Rate Variable
class InterestCalculator:
    def __init__(self, rate):
        self.rate = rate

    def calculate_interest(self, amount):
        return amount * self.rate

# Example usage
interest_calculator = InterestCalculator(0.1)
print(interest_calculator.calculate_interest(1000))

# Test Cases for Refactored Code without Global Variables

def test_calculate_interest():
    assert calculate_interest(1000, 0.1) == 100.0
    assert calculate_interest(2000, 0.05) == 100.0
    assert calculate_interest(500, 0.2) == 100.0

# Test Cases for Refactored Code using a Class to Encapsulate the Rate Variable
def test_interest_calculator():
    interest_calculator = InterestCalculator(0.1)
    assert interest_calculator.calculate_interest(1000) == 100.0
    interest_calculator.rate = 0.05
    assert interest_calculator.calculate_interest(2000) == 100.0
    interest_calculator.rate = 0.2
    assert interest_calculator.calculate_interest(500) == 100.0

# Run Test Cases
test_calculate_interest()
test_interest_calculator()
print("All test cases passed!")
```

Output:

```
100.0
100.0
All test cases passed!
```

Task Description #2 : (Refactoring Deeply Nested Conditionals)

- Task: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.
- Focus Areas: Readability

  Logical simplification

  Maintainability

Legacy Code:

score = 78

if score >= 90:

print("Excellent")

else:

if score >= 75:

print("Very Good")

else:

if score >= 60:

print("Good")

else:

print("Needs Improvement")

code screenshot:

```python
score = 78

if score >= 90:
    print("Excellent")
elif score >= 75:
    print("Very Good")
elif score >= 60:
    print("Good")
else:
    print("Needs Improvement")

# Alternatively, using a mapping-based approach
score = 78
grade_mapping = {
    range(90, 101): "Excellent",
    range(75, 90): "Very Good",
    range(60, 75): "Good",
    range(0, 60): "Needs Improvement"
}
for score_range, grade in grade_mapping.items():
    if score in score_range:
        print(grade)
        break

# also provide the test cases for the refactored code: ...
# Test Case 1: Score = 95 (Expected Output: "Excellent")
score = 95
if score >= 90:
    print("Excellent")
elif score >= 75:
    print("Very Good")
elif score >= 60:
    print("Good")
else:
    print("Needs Improvement")

# Test Case 2: Score = 80 (Expected Output: "Very Good")
score = 80
if score >= 90:
    print("Excellent")
elif score >= 75:
    print("Very Good")
elif score >= 60:
    print("Good")
else:
    print("Needs Improvement")

# Test Case 3: Score = 65 (Expected Output: "Good")
score = 65
```

```python
# Test Case 3: Score = 65 (Expected Output: "Good")
score = 65
if score >= 90:
    print("Excellent")
elif score >= 75:
    print("Very Good")
elif score >= 60:
    print("Good")
else:
    print("Needs Improvement")

# Test Case 4: Score = 50 (Expected Output: "Needs Improvement")
score = 50
if score >= 90:
    print("Excellent")
elif score >= 75:
    print("Very Good")
elif score >= 60:
    print("Good")
else:
    print("Needs Improvement")
```

Output:

```
Very Good
Very Good
Excellent
Very Good
Good
Needs Improvement
```

Task 3 (Refactoring Repeated File Handling Code)

- Task: Use AI to refactor repeated file open/read/close logic.
- Focus Areas:

  DRY principle

  Context managers

  Function reuse

Legacy Code:

```
f = open("data1.txt")
print(f.read())
f.close()
f = open("data2.txt")
print(f.read())
```

f.close()

Expected Outcome:

o Reusable function using with open() and parameters.

```python
def read_file(file_name):
    with open(file_name, 'r') as f:
        return f.read()

# Example usage
print(read_file("data1.txt"))
print(read_file("data2.txt"))
```

Task 4 (Optimizing Search Logic)

• Task: Refactor inefficient linear searches using appropriate

data structures.

• Focus Areas:

o Time complexity

o Data structure choice

Legacy Code:

users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ")

found = False

for u in users:

if u == name:

found = True

print("Access Granted" if found else "Access Denied")

Expected Outcome:

o Use of sets or dictionaries with complexity justification.

Code screenshot:

```python
# Refactored Code using a Set for O(1) average time complexity
users = {"admin", "guest", "editor", "viewer"}  # Using a set
name = input("Enter username: ")
if name in users:  # O(1) average time complexity for set membership test
    print("Access Granted")
else:
    print("Access Denied")

# Refactored Code using a Dictionary for O(1) average time complexity
users = {"admin": True, "guest": True, "editor": True, "viewer": True}  # Using a dictionary
name = input("Enter username: ")
if name in users:  # O(1) average time complexity for dictionary key lookup
    print("Access Granted")
else:
    print("Access Denied")

# Test Case 1: Valid username
# Input: "admin"
# Expected Output: "Access Granted"
# Test Case 2: Invalid username
# Input: "user"
# Expected Output: "Access Denied"
# Test Case 3: Empty username
# Input: ""
# Expected Output: "Access Denied"
# Test Case 4: Case sensitivity check
# Input: "Admin"
# Expected Output: "Access Denied" (assuming case-sensitive usernames)

# Test Case 5: Another valid username
# Input: "editor"
# Expected Output: "Access Granted"
```

Output:

```
Enter username: Rahil Raza
Access Denied
Enter username: admin
Access Granted
```

Task 5 (Refactoring Procedural Code into OOP Design)

• Task: Use AI to refactor procedural code into a class-based design.

• Focus Areas:

o Object-Oriented principles

o Encapsulation

Legacy Code:

salary = 50000

tax = salary * 0.2

net = salary - tax

print(net)

Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and attributes.

Code screenshot:

```python
class EmployeeSalaryCalculator:
    def __init__(self, salary):
        self.salary = salary

    def calculate_tax(self):
        return self.salary * 0.2

    def calculate_net_salary(self):
        tax = self.calculate_tax()
        return self.salary - tax
# Example usage
employee = EmployeeSalaryCalculator(50000)
net_salary = employee.calculate_net_salary()
print(net_salary)

# Test Case 1: Basic Salary Calculation
employee1 = EmployeeSalaryCalculator(50000)
assert employee1.calculate_tax() == 10000, "Test Case 1 Failed: Tax calculation is incorrect."
assert employee1.calculate_net_salary() == 40000, "Test Case 1 Failed: Net salary calculation is incorrect."

# Test Case 2: Zero Salary
employee2 = EmployeeSalaryCalculator(0)
assert employee2.calculate_tax() == 0, "Test Case 2 Failed: Tax calculation for zero salary is incorrect."
assert employee2.calculate_net_salary() == 0, "Test Case 2 Failed: Net salary calculation for zero salary is incorrect."
# Test Case 3: Negative Salary
employee3 = EmployeeSalaryCalculator(-50000)
assert employee3.calculate_tax() == -10000, "Test Case 3 Failed: Tax calculation for negative salary is incorrect."
assert employee3.calculate_net_salary() == -40000, "Test Case 3 Failed: Net salary calculation for negative salary is incorrect."
print("All test cases passed!")
```

Output:

```
40000.0
All test cases passed!
```

Task 6 (Refactoring for Performance Optimization)

• Task: Use AI to refactor a performance-heavy loop handling

large data.

• Focus Areas:

o Algorithmic optimization

o Use of built-in functions

Legacy Code:

total = 0

for i in range(1, 1000000):

if i % 2 == 0:

total += i

print(total)

Expected Outcome:

o Optimized logic using mathematical formulas or comprehensions, with time comparison.

Code screenshot:

```python
# Refactored Code using Mathematical Formula
n = 999998  # Largest even number less than 1,000,000
total = n * (n + 2) // 4  # Sum of first n even numbers
print(total)
# Refactored Code using List Comprehension
total = sum(i for i in range(1, 1000000) if i % 2 == 0)
print(total)
# Time Comparison
import time
# Timing the original code
start_time = time.time()
total = 0
for i in range(1, 1000000):
    if i % 2 == 0:
        total += i
end_time = time.time()
print(f"Original code time: {end_time - start_time:.6f} seconds")
# Timing the mathematical formula code
start_time = time.time()
n = 999998
total = n * (n + 2) // 4
end_time = time.time()
print(f"Mathematical formula code time: {end_time - start_time:.6f} seconds")
# Timing the list comprehension code
start_time = time.time()
total = sum(i for i in range(1, 1000000) if i % 2 == 0)
end_time = time.time()
print(f"List comprehension code time: {end_time - start_time:.6f} seconds")
```

Output:

```
Mathematical formula code time: 0.000000 seconds
List comprehension code time: 0.090078 seconds
```

Task 7 (Removing Hidden Side Effects)

• Task: Refactor code that modifies shared mutable state.

• Focus Areas:

o Functional-style refactoring

o Predictability

Legacy Code:

data = []

def add_item(x):

data.append(x)

add_item(10)

add_item(20)

print(data)

Expected Outcome:

o Refactored function returning values instead of mutating globals.

Code screenshot:

```
def add_item(data, x):
    return data + [x]
data = []
data = add_item(data, 10)
data = add_item(data, 20)
print(data)
```

Output:

```
PS C:\Users\
[10, 20]
```

Task 8 (Refactoring Complex Input Validation Logic)

• Task: Use AI to simplify and modularize complex validation

rules.

• Focus Areas:

o Readability

o Testability

Legacy Code:

password = input("Enter password: ")

if len(password) >= 8:

if any(c.isdigit() for c in password):

if any(c.isupper() for c in password):

print("Valid Password")

else:

print("Must contain uppercase")

else:

print("Must contain digit")

else:

print("Password too short")

Expected Outcome:

o Separate validation functions with clear responsibility

code screenshot:

```python
def is_valid_length(password):
    return len(password) >= 8

def has_digit(password):
    return any(c.isdigit() for c in password)

def has_uppercase(password):
    return any(c.isupper() for c in password)

password = input("Enter password: ")
if is_valid_length(password):
    if has_digit(password):
        if has_uppercase(password):
            print("Valid Password")
        else:
            print("Must contain uppercase")
    else:
        print("Must contain digit")
else:
    print("Password too short")
```

Output:

```
Enter password: 1434
Password too short
```

Task 8 (Refactoring Complex Input Validation Logic)

• Task: Use AI to simplify and modularize complex validation rules.

• Focus Areas:

o Readability

o Testability

Legacy Code:

password = input("Enter password: ")

if len(password) >= 8:

if any(c.isdigit() for c in password):

if any(c.isupper() for c in password):

print("Valid Password")

else:

```
print("Must contain uppercase")
else:
print("Must contain digit")
else:
print("Password too short")
```

Expected Outcome:

o Separate validation functions with clear responsibility

code screenshot :

sers > phani > OneDrive > Desktop > Btech > 3-2 > assign1.1.py > ...

```python
def is_valid_length(password):
    return len(password) >= 8

def has_digit(password):
    return any(c.isdigit() for c in password)

def has_uppercase(password):
    return any(c.isupper() for c in password)
def validate_password(password):
    if not is_valid_length(password):
        return "Password too short"
    if not has_digit(password):
        return "Must contain digit"
    if not has_uppercase(password):
        return "Must contain uppercase"
    return "Valid Password"
password = input("Enter password: ")
validation_result = validate_password(password)
print(validation_result)

# Test cases for the password validation functions
def test_is_valid_length():
    assert is_valid_length("password") == True
    assert is_valid_length("pass") == False

def test_has_digit():
    assert has_digit("password1") == True
    assert has_digit("password") == False

def test_has_uppercase():
    assert has_uppercase("Password") == True
    assert has_uppercase("password") == False

# Run the tests
test_is_valid_length()
test_has_digit()
test_has_uppercase()
print("All tests passed!")
```

Output screenshot:

```
Enter password: saisharan
Must contain digit
All tests passed!
PS D:\Devops>
```